

INSTITUTO SUPERIOR TÉCNICO

Departamento de Engenharia Informática

Forensics Cyber-Security

MEIC, METI

Digital Forensics Report

Lab Assignment II – Operation DeepFISH – Stage II

Group:	53	
Student 1:	João Sérgio Abreu Viegas	109685
Student 2:	Rafael de Oliveira Gomes	110285
Student 3:	Ilès Nassim Ouyahia	121299

2026/2027

1 Acquired artifacts

We group the artifacts into (a) the evidence seized and provided for Stage II, (b) the artifacts recovered from the BitLocker-protected `D:` volume, (c) the registry hives extracted from the `C:` system volume, and (d) the artifacts recovered from the Linux workstation. Hashes of recovered artifacts are the SHA-256 of the exact bytes extracted; see Section 2 for how each was obtained.

Seized evidence (Stage II)

Name	Provenance	SHA-256 Value
`windows_laptop_memory.raw`	RAM capture, Windows laptop	`c0b66a9ec5ff16d8c01726a8717648f729857fa059ba7485c5c1717587892be3`
`windows_laptop_disk.dd`	Disk image, Windows laptop	`5eca1f65df49ccf5bddeb8c5c3a44fe1a8a9e6faab04ed601d565b915bbaa6a8`
`linux_workstation_disk.dd`	Disk image, Linux workstation	`b31261098d901de0be201f4f78f5816c946490b9eb4ae2c94dc2fcf4af4e3368`

Recovered artifacts (Windows laptop, `D:` volume)

Name	Provenance	SHA-256 Value
`dislocker-file` (decrypted `D:`)	Decrypted `D:` volume	not submitted due to size, can be regenerated with `k3_xts128.fvek`
`k3_xts128.fvek`	FVEK from RAM	`9f113cf3f7ee5d3cb6638a3f5a5bad2006667776a9971224cf55efaf56dfda5c`
`ConsoleHost_history.txt`	`johnny` PSReadLine	`53dcd531d211c2a219df6ff859357c9ce018296e53106a0b097c9d3356314372`
`discord_cache_1975/76/...`	`johnny` Discord cache	*(see manifest, Section 5)*
`Opera_History.sqlite`	`johnny` Opera GX profile	`3890e8ed751cb39632ee7a73461db812745b98e768a40b3ba7b9717aee15cf8e`
`TranscodedWallpaper`	`johnny` Themes	`422b5f86c78905a5ff5707ce7ba0cd76a197e771dfa142db89ed25922479fcc2`
`johnny_NTUSER.DAT`	`johnny` user hive	`79802cc2853d4092d2c94f159f4cb9bf14ba27398adf8de71a477126f468e299`
`mom-photo.jpg`	`johnny` Pictures	`c323d401bd25fd8a6ae6e3b8f7dba724ce95d51c886b15653a2f5a0863c800c8`
`hate-this-place.jpg`	`johnny` Pictures	`fe8f99ce75f92a4aee463d5b535d2737e0e042d8bfb59bf44d123c9704f6be55`
`Natural_Language_MAP1.pdf`	`johnny` Desktop (`Exams_LN`)	`fe1a5720a55ccacfb85d90b12bf7c7b7a745220a5c8d8c0621806546277174d1`

Registry hives (Windows laptop, `C:` volume)

Name	Provenance	SHA-256 Value
`SYSTEM.hive`	`C:System32\config\SYSTEM`	`4b7466fdcc91b00c622ac995d85af3872efed87654072b3ed78121e20b7bf4b4`
`SAM.hive`	`C:System32\config\SAM`	`0e0d9235598d1c67445eabd2b8d151afce3820a32123550f68e8e9442129797f`

Name	Provenance	SHA-256 Value
SECURITY.hive	C:\System32\config\SECURITY	fd12b21d9f979bf7709cefe36d858ff1217d66b8d7ccd393d772fe30895d5c42
SOFTWARE.hive	C:\System32\config\SOFTWARE	af5dbd1818a94b6041a9f76b38f7f067e46fe8a47b72ac46e048dc8de58c2209

Recovered artifacts (Linux workstation)

Name	Provenance	SHA-256 Value
stash.zip	johnnymusk backups/	d0544dcbcbccbce302ad5f811de9f59ee786d94e7b311b2d3ae6f3b153a0c308
bash_history	johnnymusk home	8aa347a60c42893f9fca7e34e6a85228a021e50e9393fe11cb2c9cde150f5d0e
autosync.sh	johnnymusk myscripts/	9f23ccbda4fcc9f3fe09f7e44736743d3e67713d6a65608975d0f2c187c60487
automerge.sh	johnnymusk myscripts/	28deaf4d9def61acfe33513e86f43d0d4122a0f34e13a69e3ceb08d5d7626aab
deploy.sh	johnnymusk myscripts/	b9d1f803f23aa66c512a8b1c5c7bd818471a2137d1632353e02f183bb3b519a5
bashrc	johnnymusk home	f469b67d52444466804217d422e2230afb9fbd19ca10dbc56ec05b9762f764aa
profile	johnnymusk home	28b4a453b68dde64f814e94bab14ee651f4f162e15dd9920490aa1d49f05d2a4
ssh_config	johnnymusk .ssh/	5d0995bbddf7472bbadcf9e8bd322e0104a20f5b1aa455bac6057aa43375ec41
johnny.pub	johnnymusk .ssh/	56eca56560d8047544c39d5b40176c4d887091ce8e810fd24611650e6d66a3a8
todo-list.txt	johnnymusk Documents/	2cbf3502525442df287da87eba76f58ef72bcd8f911723ff1839f2a0c4a9a126
apt_history.log	/var/log	ef2c560039d500fe0424d887254027cce02dcc1269336446dbe8cd3d3f60a1b9
auth.log	/var/log	d741fb1da6229600107fb47077199e9abc93f8bcb885fab2b8c4e94cce222421
dpkg.log	/var/log	443711026b5d8d14cfd5e33394fbb1e850aa947f10b66dadb6a739051f461948
kern.log	/var/log	80ab35dfed5d2bfd74cbf4ab1b2e13036339743b7441999c03f3ea61e01e6af6
syslog	/var/log	81acba25682bd75abbc1751bbc1e1f4235631ddd3c02bf26da25015a6316f787
journal_export.txt	/var/log/journal	6c0b206cb13f81da689fe8ed8deb3f2105928eaf28f0acc14aeb9d692f994937

2 Report of all findings (Investigation and Evidence Analysis)

2.1 Evidence integrity and acquisition context

Upon analyzing the live-memory image `windows_laptop_memory.raw`, `windows.info` (Volatility 3 Framework 2.28.2) reports a Windows 10 build 19041 x64 host, computer name `JOHNNY-PC`, with a system time of **2026-08-25 21:03:25 UTC**. All Volatility timestamps in this report are UTC. Windows disk timestamps are used without adjustment, as instructed; only the Linux workstation image is subject to the −27-day correction.

The process listing shows the machine had been running for roughly three minutes at capture time and that **no interactive user session was present** (no `explorer.exe`; only `winlogon.exe`/`LogonUI.exe` in session 2). The only user activity in memory is a chain of Windows Remote Management processes: `svchost.exe` → three `winrshost.exe` instances → `cmd.exe` (PID 4416) → `powershell.exe` (PID 3144), created 21:02:40–21:02:57 UTC. `windows.cmdline` shows PID 4416 ran a Base64/UTF-16LE `-encodedcommand`; decoded with

```
grep -P '^4416' cmdline.txt | grep -oP 'encodedcommand \K\S+' \
  | base64 -d | iconv -f UTF-16LE -t UTF-8
```

it is a **cleanup script executed by the acquisition process**, not suspect activity: it removes WinRM setup logs and `C:\Windows\Panther\Unattend*`, reverts WinRM configuration, and deletes the listener. `windows.netscan` confirms the controlling host was `10.0.2.2` (the virtualization gateway), connecting to `10.0.2.15:5985`.

Observation: files absent from these paths on the disk image are explained by this investigator-side activity. The script's own comments further state that the BitLocker passphrase for `D:` was deliberately left resident in memory and that the intended recovery path is extracting the FVEK from RAM.

2.2 Disk layout and recovery of the BitLocker `D:` volume

`mmls windows_laptop_disk.dd` reports a GPT disk with the Windows system volume (`C:`) at sector 239616, a 750 MB NTFS volume at 41199616, and a third volume at **sector 42735616**. Reading the first sector of each candidate,

```
dd if=windows_laptop_disk.dd bs=512 skip=42735616 count=1 2>/dev/null \
  | xxd | head -1
```

returns the signature `-FVE-FS-`, identifying the volume at 42735616 as the BitLocker-encrypted `D:`.

We recovered the Full Volume Encryption Key from RAM with the `windows.bitlocker.BitlockerFVEKScan` Volatility plugin, which returned three candidate keys. We tested each against `D:` and verified the result with TSK:

```
vol -f windows_laptop_memory.raw -p ./plugins -o out/fvek \
    windows.bitlocker.BitlockerFVEKScan --tags FVEc Cngb None --dislocker
sudo dislocker -r -V windows_laptop_disk.dd -O $((42735616*512)) \
    -k k3_xts128.fvek -- /mnt/bl
sudo fsstat /mnt/bl/dislocker-file   # File System Type: NTFS, Volume Name "Data"
```

Only the AES-XTS-128 key at pool offset `0xa78da5857000` (FVEK `d7fc6f8caa41cf2a4a1ab094fe198b04`, tweak `779232c3b23521f858350968b87d1d31`) produced a valid NTFS volume; the other candidates failed the `fsstat` check. Decryption was read-only. All subsequent `D:` analysis used TSK (`fls`, `istat`, `icat`) on `dislocker-file`.

2.3 User attribution and profile

The `D:` volume ("Data") holds the user profile `johnny` (SID `S-1-5-21-2191727074-1144793430-742902354-1000`); the Windows system resides on `C:`, which explains why this user's `NTUSER.DAT` was not resident in the memory image. The wallpaper (`TranscodedWallpaper`) is an image of the character **Elliot** from the series *Mr. Robot*. The account owner corresponds to the person of interest identified in Stage I (João Musk, online alias `MuskyBoi` / `Johnny`).

The `johnny` user hive (`johnny_NTUSER.DAT`, parsed with `regripper -a`) corroborates this account. `LogonStats` records a first logon of **2026-07-28 20:11:17Z**, matching the operator's own statement, in the Discord cache, of a "fresh windows installation" that same night. The `Nation` key reports region Portugal (Geo 193). The autostart `Run` key lists only `Discord` (from `D:\Users\johnny\AppData\Local\Discord\Update.exe`) and `Opera GX`, and the recovered Desktop jump-list references `Opera GX Browser.lnk`, `Discord.lnk`, and the `Exams_LN` folder.

Observation: this is consistent with the recent (2026-07-28) clean installation rather than, by itself, evidence of wiping.

2.4 Traces of the phishing campaign

PowerShell history

`ConsoleHost_history.txt` contains three commands, the first of which links the laptop directly to the Stage I phishing infrastructure:

```
ssh muskyboi@146.193.41.109 "cat ~/fenix/peixinho/credentials.txt"
ssh ist101337@sigma.ist.utl.pt
scp ist101337@sigma.ist.utl.pt:~/Linguagem_Natural/Exams/
Natural_Language_MAP1_2026-2027.pdf D:\Users\johnny\Desktop\Exams_LN\
```

The user `muskyboi` reads a `credentials.txt` under `~/fenix/peixinho/` on an IST/INESC-range host (`146.193.41.109`). `fenix` and `peixinho` ("little fish") correspond to the cloned Fénix login page and the "Deep Fish" operation; the remaining two commands exfiltrate a Natural Language exam (`Natural_Language_MAP1_2026-2027.pdf`) from `sigma.ist.utl.pt`.

Browser history

Parsing the Opera GX history (`Opera_History.sqlite`) with

```
sqlite3 Opera_History.sqlite \
 "SELECT datetime(last_visit_time/1000000-11644473600,'unixepoch'),url,title \
  FROM urls ORDER BY last_visit_time;"
```

shows a progression on the user's own profile (times UTC): searches for "how does phishing works" and "social engineering" (2026-08-10 10:00–10:05), ChatGPT queried for "how to create a phishing website" (10:50), then on 2026-08-19 a search for "how to send email as someone else" (10:30) immediately followed by a visit to `emkei.cz` (10:37), a public spoofed-email service consistent with the Stage I phishing emails.

2.5 Concealed communications and file provenance

The Discord cache under the `johnny` profile preserves message history in JSON (recovered by inode with `icat`) and one Opus voice message (`discord_cache_1977`, 75.9 s).

A direct-message thread with user `DuckyDonald` (July–August 2026) records `DuckyDonald` describing an internship at **AMA** handling "AI stuff", he reports the sudden appearance of incriminating files on his drive, and a request that `johnny` hold those files in the voice message. He states he left a USB drive in a "Locky" locker at Alameda Metro and provides scripts to conceal the files "inside the personal stuff".

On 2026-08-25 (`discord_cache_1979`, `discord_cache_1980`) `johnny` confirms:

> "I ran each script on the file it matched, so everything is hidden inside the personal files now. I put it all together in a zip with a password and saved it on my linux dev machine."

and, critically for provenance:

> "That folder is a repo. I have a script that commits and pushes it every time I open a terminal … Every time I hid one of your files and saved it, the sync took it with the rest."

Forensic inference: each hidden file was steganographically embedded into a personal carrier file (the Stage I source artifacts of Table 1), and an automatic synchronisation of a personal Git repository published the carriers, with their concealed payloads, so that they reached the operator's phishing-site repository recovered in Stage I. The Discord wording ("a script that commits and pushes") is the operator's informal description; the exact mechanism (an autosync script invoked from `.bashrc`,

followed by a server-side merge) is established directly on the Linux workstation in Section 2.10, so the mechanism is treated there as a direct observation rather than an inference.

In a second (group) channel, `johnny` states on 2026-08-25 that `DuckyDonald` is in fact **"Tiago Sintaxe"**, the teaching assistant he had phished, and describes an intent to destroy the USB. These attributions and their interpretation are developed in Section 3.

2.6 Anti-forensic activity (Windows laptop)

The Discord thread records explicit intent to destroy evidence: instructions to "delete the loose files and run the secure delete tool", and `johnny`'s statements "I'm going to `srm` the whole repos folder", "I deleted everything", and a plan to throw the USB "in the Tejo". On the recovered `D:` volume, the `$RECYCLE.BIN` for the `johnny` SID contains only `desktop.ini`, consistent with an emptied Recycle Bin.

2.7 The `C:` system partition and the protected-archive passphrase

The `C:` system volume (NTFS, sector 239616) is not encrypted. We extracted the registry hives read-only, by inode, with TSK:

```
C=windows_laptop_disk.dd; OFF=239616
fls -o $OFF -r -p $C | grep -iE 'System32/config/(SAM|SYSTEM|SECURITY|SOFTWARE)$'
icat -o $OFF $C 40073 > SAM.hive        # Windows/System32/config/SAM
icat -o $OFF $C 40354 > SYSTEM.hive     # Windows/System32/config/SYSTEM
icat -o $OFF $C 40075 > SECURITY.hive   # Windows/System32/config/SECURITY
icat -o $OFF $C 44810 > SOFTWARE.hive   # Windows/System32/config/SOFTWARE
```

Account password: A first `samdump2 SYSTEM.hive SAM.hive` reported the `johnny` account with the null-password NT hash `31d6cfe0d16ae931b73c59d7e0c089c0`; this was an artifact of deriving the syskey without the SECURITY hive. Re-running with the boot key (`ae58242038baaa94990f0135cb7b90e3`, from SYSTEM) resolves the true hash:

```
pypykatz registry SYSTEM.hive --sam SAM.hive --security SECURITY.hive
```

which reports `johnny:1000:…:fb06de73293e2350d5087dc306a52934:::`. The `SOFTWARE` hive's Winlogon key shows automatic logon is enabled (`AutoAdminLogon = "1"`, `DefaultUserName = "johnny"`, `AutoLogonSID` = the `johnny` SID). The corresponding cleartext password is stored, as is standard for autologin, as the `DefaultPassword` LSA secret in the SECURITY hive; decrypting it with the boot key yields:

```
=== LSA Default Password ===
Username: UNKNOWN
Password: fsociety
```

Observation: the `johnny` account password is `fsociety` (another reference to the *Mr. Robot* series). The identical NT hash on the built-in `Administrator` account indicates it shares the same password.

The handwritten note and the protected archive: The note recovered during the search (assignment Figure 1) specifies the passphrase of the password-protected archive as the SHA-256 of five clues, in lowercase and joined by underscores. Each clue resolves to evidence recovered above:

#	Clue (handwritten note)	Resolved value
1	The laptop's name	`johnny-pc`
2	First name of the person on the wallpaper	`elliot` (*Mr. Robot* wallpaper)
3	The region in the "JJK" picture on the laptop	`shibuya` (`hate-this-place.jpg`, the Shibuya Station scene from *Jujutsu Kaisen*)

#	Clue (handwritten note)	Resolved value
4	The Windows password	`fsociety` (`DefaultPassword` LSA secret)
5	The spice naming the restaurant in the photo of Fábrica da Pólvora	`pimenta` (restaurant *Maria Pimenta*)

Concatenating the tokens and hashing them reproduces the archive passphrase:

```
echo -n "johnny-pc_elliot_shibuya_fsociety_pimenta" | sha256sum
# ab99749a576b8070ddf665ce139afd43e7c2a3f5b55b3173e98791c281c1454d
```

Note: the password-protected archive itself was not stored on the seized Windows laptop. The passphrase derived here is confirmed against that archive on the Linux workstation in Section 2.9.

2.8 The Linux workstation

The image `linux_workstation_disk.dd` was analysed read-only with The Sleuth Kit (`mmls`, `fsstat`, `fls`, `istat`, `icat`), `7z` and `git`, and every extracted file was SHA-256 hashed (Section 1). `mmls` shows an ext4 partition at **sector 2203648**. The system is **Ubuntu 24.04 ARM64** running as a VMware virtual machine, hostname **`johnny-dev`** (the "linux dev machine" of the Discord cache), with a single interactive account, **`johnnymusk`** (uid 1000, member of `sudo`).

As a consistency check, the logged `apt install secure-delete` maps to **2026-08-25 16:02:23 WEST** (following the −27-day rule), inside the 14:16-18:10 login session that also contains every git commit of that day.

2.9 The protected archive and the embedding session

`/home/johnnymusk/backups/stash.zip` was created from `~/ducky/hidings/artifacts/` (`.bash_history` line 69) with

```
zip -r -e -m ~/backups/stash.zip .
```

`-e` encrypts with legacy ZipCrypto (`7z l -slt` reports `ZipCrypto Deflate`) and `-m` deletes the plaintext copies. The archive opens with the passphrase derived from the handwritten note, confirming that the zip was saved "on my linux dev machine".

It holds nine files: eight carriers and `subtitles.srt`, a helper used only to encode `myzip.zip`. The carrier hashes match the Stage I Table 1 values (full hashes in Section 5). The mapping comes from the embedding commands in `.bash_history`:

Carrier (in `stash.zip`)	Hidden secret	Embedding tool
`blank`	`reddit.png`	`create.sh`
`polish_cow.webp`	`amelia_tools_config.json`	`webp_duration_encode.py`
`myzip.zip`	`amelia.pdf`	`create-zip.sh`
`subtitles.srt`	(helper for `myzip.zip`)	`wordlist_encode.py`
`Grand Theft Auto VI Trailer 1.mp4`	`Technical_Assurance.pdf`	`polyglot_create.sh`
`ESP32_ADXL345_3Axis_Telemetry.xlsx`	`train_anisotropic_override.py`	`excel_lsb_encode.py`
`gta_v_iceberg_unexplained.jpg`	`ama_api_tokens.json`	`dct_encode.py`
`dj_cara_after_hours.wav`	`rag_poisoning_patch.diff`	`wav_encode.py`
`[Erai-raws] Shingeki no Kyojin - 01 [...].mkv`	`telegram_chat.webp`	`video_encode.py`

The rest of `.bash_history` gives the order of events:

- lines 7-11: a USB volume labelled **`TEARSINRAIN`** was mounted and copied to `~/ducky/`;
- lines 14-39: each secret file was opened and the hiding tools in `aux/` were made executable;
- lines 41-55: one embedding command per carrier, each overwriting a file already tracked in `~/repos/personal`;
- lines 59-63: three files removed from the repo's `Downloads/`, matching the three deletions in the `sync` commit `3483ea7e` (15:42:13), so the embedding took place before that time;
- lines 56 and 64-69: the carriers were copied to `~/ducky/hidings/artifacts/` and zipped.

`Documents/todo-list.txt` lists "Check USB drive ducky sent me" and "back up peixinho", and states "LN exam is on my windows bitlocker drive", which matches the BitLocker `D:` volume on the laptop.

2.10 How the carriers reached the phishing repository

Automatic commits

`.bashrc` (lines 119-122) starts `autosync.sh` in the background, with all output discarded, in every interactive shell:

```
(sh ~/myscripts/autosync.sh &) >/dev/null 2>&1
```

`autosync.sh` runs `git add -A; git commit` in `~/repos/personal` and picks the message by rule (only deletions: `sync`; only `Downloads/`: `sync downloads`; only `Documents/`: `telemetry files`; otherwise `phone backup`). The four personal-repo commits of 25 August follow these rules exactly. No Git hooks are installed and `.bash_history` has no manual commit in that repo.

Merge into the phishing repository

`automerge.sh` and `.bash_history` line 84 run

```
ssh fenix-prod \
  'cd ~/fenix && git fetch upstream && git merge --no-ff --no-edit upstream/main'
```

whose default message is that of the merge commit `7fc49691` (16:03:11). `.ssh/config` resolves `fenix-prod` to **`muskyboi@146.193.41.109`** (key `~/.ssh/muskyboi`), the same host and account as in the Windows PowerShell history. On the server, `upstream` is the personal repository `muskyboi-archive/personal` (Stage I), so the merge brought the carriers into the phishing-site repository, whose working tree is the public webroot with an exposed `.git/`. `deploy.sh` deploys `MuskyBoi/peixinho` to the same host.

Both SSH private keys are unencrypted and carry the comment `sussyjohnnyhacker@gmail.com`, the same address as the Git author. Only their hashes (Section 1) and public-key fingerprints are reported:

`muskyboi` : `SHA256:osh80EJDQuHwcEqcdryNBuqFnwh1wnhcSfAPfkNggJo`,

`johnny` : `SHA256:k5dMFQvwbsXib/+A+Gnr1AoaJmzSainr7K1z9lmcDSU`.

Cleanup: `.bash_history` line 86 runs, on the server, `git rm -r Downloads Documents Pictures Music Videos notes` and `git commit -am cleanup` (commit `e87b2eed`, 18:05:37). This removed the files from the latest revision only.

2.11 Anti-forensic activity and limitations (Linux workstation)

Secure deletion: `.bash_history` lines 74-81 show `srm -ll -r ~/ducky` failing, then `sudo apt install secure-delete` and `srm -ll -r` on `~/ducky` and `~/repos`. The installation is logged in `auth.log`, `apt/history.log` and `dpkg.log` at **2026-08-25 16:02:23-25 WEST**. `srm` overwrites data before unlinking, so neither `~/ducky` nor `~/repos/personal` leaves an entry in `fls -rd` (its 2,681 deleted entries are all package-manager files).

Journal wipe: `auth.log` records `sudo journalctl --rotate` and `journalctl --vacuum-time=1s` by `johnnymusk` on **2026-08-24 23:30 WEST**, so no journal survives from before that time.

The USB `TEARSINRAIN` appears in no surviving log (the journal, `kern.log` and `syslog` show only VMware virtual devices), so no serial number is available to compare with the locker drive; `.bash_history` is the only evidence of it.

2.12 Reproducibility and timestamp sources

All recovered artifacts were extracted read-only and hashed (Section 1, Section 5).

Windows timestamps come from Volatility (UTC), NTFS MAC times (`mactime -z UTC`) and Discord records (`+01:00`, as stored), with no conversion applied. Linux timestamps follow the assigned −27-day rule. The consolidated timeline is given in the Timeline deliverable.

3 Analysis of relevant findings

3.1 Phishing website and recovered artifacts

Do the seized systems contain traces of the phishing website and/or the hidden files recovered during Lab Assignment I? If so, identify the relevant evidence, its location, and its SHA-256 value where applicable.

Both are present on the seized systems. The analysis in Section 2 ties the Stage I phishing infrastructure and the Stage I hidden files to concrete artifacts on all three images.

Analysis - traces of the phishing website

Windows laptop: The PowerShell history `ConsoleHost_history.txt` (Section 1) contains `ssh muskyboi@146.193.41.109 "cat ~/fenix/peixinho/credentials.txt"`. The host `146.193.41.109`, the account `muskyboi`, the `~/fenix/peixinho/` path and the `credentials.txt` harvested-credential store all correspond to the Stage I “Deep Fish” site. The Opera GX history `Opera_History.sqlite` records the phishing-preparation activity (searches on phishing and social engineering, a ChatGPT query on “how to create a phishing website”, and a visit to the spoofed-mail service `emkei.cz`).

Linux workstation: `deploy.sh` deploys `git@gitlab.com:MuskyBoi/peixinho.git` to `fenix-prod`; `.ssh/config` resolves `fenix-prod` to `muskyboi@146.193.41.109`, the same host and account as the Windows history. `automerge.sh` performs the server-side merge into `~/fenix`, and the local clone `~/backups/peixinho` carries the single remote `origin = https://fenix.dpss.inesc-id.pt/.git/`, i.e. the exact exposed `.git/` that Stage I dumped. These place the development, deployment and maintenance of the phishing site on this workstation.

Analysis - the hidden files

The password-protected archive `stash.zip` holds the eight stego carriers plus the `subtitles.srt` helper (Section 2.9). The carrier SHA-256 values match, byte-for-byte, the Stage I Table 1 artifacts recovered from the phishing repository; the full hashes are in Section 5. The same carriers are therefore present on the Linux image (inside `stash.zip`) and, through the Git history recovered in Stage I, in the phishing-site repository.

Forensic inference: the match of the carrier hashes across the two stages, together with the `fenix-prod`/`146.193.41.109` identity shared between the Windows history and the Linux `.ssh/config`, establishes that the Stage II machines are the operational source of the Stage I phishing site and of the concealed material recovered from it.

3.2 Provenance and temporal reconstruction

Can you reconstruct the provenance and handling of these files? Provide a timeline of relevant events showing how the phishing website was developed and operated, and how the hidden files were handled on the seized systems.

The provenance is reconstructed in detail in Section 2.9 and Section 2.10; the consolidated chronology is in the separately submitted Timeline deliverable. The summary below separates what is directly observed from what is bounded by inference, and states the timestamp sources and conversions used.

Analysis - handling of the hidden files

`.bash_history` records the embedding session end to end: the USB volume `TEARSINRAIN` was copied into `~/ducky/` (lines 7–11); each secret file was opened (lines 14–24); one embedding command was run per carrier, overwriting the carrier in place (lines 41–55, mapping in Section 2.9); the nine files were then collected into `~/ducky/hidings/artifacts/` and sealed into `~/backups/stash.zip` with `zip -r -e -m` (lines 64–69). The archive opens with the passphrase derived from the handwritten note (Section 2.7), confirming the Discord claim that it was “saved on my linux dev machine”.

Analysis - how the carriers reached the public site

`.bashrc` launches `autosync.sh` in every interactive shell; that script’s `git add -A; git commit` in `~/repos/personal` produced the four personal-repo commits of 25 August whose rule-chosen messages match exactly the commits of that day. The server-side merge `7fc49691` (16:03:11) pulled `upstream (muskyboi-archive/personal)` into the phishing-site repository, and the server `cleanup` commit `e87b2eed` (18:05:37) removed the files from the latest revision only, leaving the Git history intact, for us to recover in Stage I.

Analysis - development of the site

From the Stage I Git history: `757af027` (2026-08-18 17:05 +0100, “production config”) turned the CAS-clone skeleton into a deployable kit and added the `.well-known/acme-challenge/…` file, i.e. an ACME HTTP-01 challenge for a real TLS certificate on `fenix.dpss.inesc-id.pt`.

Timestamp sources and conversions

Windows/Volatility times are UTC with no adjustment; NTFS MAC times via `mactime -z UTC`; Discord records as stored (`+01:00`). Linux ext4 metadata and `/var/log` entries are shifted **−27 days** (assignment rule); `.bash_history` carries no timestamps, so only command order is known and is bracketed by the git commits and inode times. The `−27-day` mapping is corroborated by the logged `apt install secure-delete` at 2026-08-25 16:02:23 WEST, which falls inside the 14:16–18:10 WEST login session that also contains every git commit of that day.

Forensic inference: No `git push` to `muskyboi-archive` was recovered from `.bashrc`, the scripts or `.bash_history`. Because the server merge fetched `3483ea7e`, the push must have occurred between 15:42:13 and 16:03:11 on 25 August; this is an **inference** bounded to that window, not an observed command.

3.3 Anti-forensic activity

Did you find evidence of anti-forensic activity? If so, describe the techniques observed and the evidence supporting your conclusions.

Analysis - secure deletion (Linux)

`.bash_history` lines 74–81 show `srm -ll -r ~/ducky` failing, then `sudo apt install secure-delete` followed by `srm -ll -r` on `~/ducky` and `~/repos`. The installation is logged in `auth.log`, `apt/history.log` and `dpkg.log` at 2026-08-25 16:02:23–25 WEST. Because `srm` overwrites before unlinking, neither `~/ducky` nor `~/repos/personal` leaves an entry in `fls -rd` (its 2,681 deleted entries are all package-manager files).

Analysis - journal wipe (Linux)

`auth.log` records `journalctl --rotate` and `journalctl --vacuum-time=1s` run via `sudo` as `johnnymusk` on 2026-08-24 23:30 WEST, so no system journal survives from before that time.

Analysis - concealment and server-side cleanup

The carriers were sealed in a ZipCrypto password-protected archive (Section 2.9), and the server `cleanup` commit `e87b2eed` removed the merged-in files from the latest revision. On the Windows side,

the Discord cache records explicit intent to destroy evidence (`srm`, “I deleted everything”, a plan to throw the USB “in the Tejo”), and the recovered `D:` volume shows an emptied Recycle Bin.

Distinguishing observed actions from alternatives

The server `cleanup` commit is an observed anti-forensic **attempt** that **failed** in effect: because it ran on the server, Git history retained every carrier, which is how we recovered them in Stage I.

The USB `TEARSINRAIN` appears in no surviving log (only in `.bash_history`). This is a **limitation**, not proof of a deliberate log wipe, since the journal predating 24 August 23:30 does not survive and the mount may simply never have been journalled.

3.4 New discoveries and investigative interpretation

What new discoveries emerge from the Stage II evidence that were not visible during Lab Assignment I? Do these findings strengthen or weaken the attribution of the phishing attack to João Musk, or point to other relevant actors or infrastructure? Based on your analysis, what hypotheses can you formulate about the larger scheme suggested by the hidden files, and what further investigative steps would you recommend to the Polícia Judiciária?

Analysis - new discoveries

Stage I saw only the public artifacts (the exposed site, the recovered repository and the carriers). Stage II adds the operator-side context that was not visible before:

- the operational infrastructure and identity: host `146.193.41.109`, account `muskyboi`, SSH private keys with the comment `sussyjohnnyhacker@gmail.com` (the same address as the Git author), and the deploy/merge scripts;
- the concealment mechanism **as an observed fact** rather than just a Discord conversation: the `autosync.sh` + `.bashrc` auto-commit and the `upstream` merge;
- the archive passphrase derived from the handwritten note, linking the Windows note, the Discord chat and the Linux archive;
- the exfiltration of a Natural Language exam (`sigma.ist.utl.pt`, `Natural_Language_MAP1_2026-2027.pdf`), on the BitLocker `D:` volume;
- the Discord thread identifying `DuckyDonald` as **Tiago Sintaxe** (the phished teaching assistant), his AMA internship doing “AI stuff”, and a USB left in a locker at Alameda.

Attribution

The Stage II evidence reinforces attribution of the phishing attack to João Musk: the consistent alias set (`johnny` / `johnnymusk` / `MuskyBoi` / `muskyboi-archive`), the SSH-key and Git author e-mail `sussyjohnnyhacker@gmail.com`, the `fsociety` / *Mr. Robot* theming across the laptop and account password, and the note-derived passphrase that binds three independent evidence sources together.

The hidden **payloads** are not the phisher’s own material: per the Discord cache, `DuckyDonald` / Tiago Sintaxe supplied the files and the hiding tools and asked `johnny` to conceal them. So the phishing attack attributes to João Musk, while the concealed material points to Tiago Sintaxe and to AMA/AMÉLIA infrastructure as a distinct, linked thread.

Hypothesis (inference): The recovered artifacts and evidence from Stage I (a GCP service-account key for the project `ama-amelia-gov-pt`, a RAG-poisoning patch, a training-override script, and a Telegram chat) suggest a larger scheme beyond university phishing: an insider leak and possible tampering of a Portuguese e-government AI system (AMÉLIA), with the phisher’s personal repository used, knowingly or not, as a covert publication channel and the material possibly destined for an external party. This is a **hypothesis** supported by the payload identifiers and the Discord narrative, not an established conclusion.

Recommended next steps for the Polícia Judiciária:

- Image and preserve the `fenix-prod` server at `146.193.41.109`, including the harvested `credentials.txt`, `authorized_keys`, and system/SSH logs, to identify victims and recover the push timestamps the workstation no longer holds.
- Obtain GitLab server-side records for `MuskyBoi/peixinho` and `muskyboi-archive/personal` (push times, source IPs, access logs) to close the inferred 15:42–16:03 push window with primary evidence.
- Coordinate with AMA to revoke and audit the `ama-amelia-gov-pt` service account and assess the AMÉLIA system for the leaked tokens and the RAG-poisoning patch; preserve relevant access logs.
- Locate and seize the USB described in the Discord cache (reported left in a locker at Alameda, and threatened to be discarded in the Tejo), which may also hold the original unconcealed files.
- Interview Tiago Sintaxe and preserve the associated Discord and e-mail accounts (`sussyjohnnyhacker@gmail.com`).
- Preserve the exam-exfiltration evidence (`sigma.ist.utl.pt`, the recovered exam PDF) as a separate academic-integrity line of inquiry.

4 Use of Artificial Intelligence

Did you use Artificial Intelligence (AI) tools in the preparation of this report? If so, identify the tool(s) used and briefly state the purpose(s) for which they were used.

We used Claude throughout the preparation of this investigation and report, for the following purposes:

We used it to help reason through and exhaust all possibilities as well as implement parts of the technical analysis: debugging outputs, suggesting tools to analyze the disks further, and helping interpret tool output when the cause of an anomaly was not immediately obvious.

We also used it to help draft and revise the text for this report, from the commands, outputs and findings we provided, and to keep the narrative and terminology consistent across sections, as well as help us with writing Typst code.

All AI-suggested techniques, scripts and hypotheses were independently reviewed and validated by the group before being relied upon or included in this report. The findings, conclusions and final wording presented here are the group's own responsibility.

5 Appendices

Discord cache: The JSON message logs and the Opus voice message (`discord_cache_1977`) recovered from the `johnny` Discord cache; see Section 2.

File	SHA-256 Value
`discord_cache_1851`	`f63f1b0aa059c51f5286c93ada16bffb6af5f8fc9aaa24f1c75361b6e1bb85d0`
`discord_cache_1951`	`cee74887189dab0d8bd3bea37e437390bb78f639d832bf8c06426349a89c9c8f`
`discord_cache_1975`	`4368ecda87cef73720da384fd9167951547096ec85e7a99c7c7cb69698286f90`
`discord_cache_1976`	`c25c32d4c61b498624369264c216a19b2782a31868f94a7537c1cdec4fc7274b`
`discord_cache_1977` (voice)	`a4fda2c29a82d461a709cdc3ea4707651549a89d9782f3d1e696b81833edeccc`
`discord_cache_1978`	`835d330d451afb4f9d47bde6dc9f1615a24001a1a82c6a10046a306a358d2ae8`
`discord_cache_1979`	`bf0d6c3cd0d8eca1d5e538359ca90292b6230f53b63dd1bd17a18bdb677f38b6`
`discord_cache_1980`	`39f7d8562d46c0fe4ba272df373e7071684afb71654bee4080d6afa0988edc27`
`discord_cache_1981`	`81822b1efac7713c3d355cfd5aa4097483ac3e305aa83ad25de650077325d7a2`

Browser artifacts (Opera GX profile)

File	SHA-256 Value
Opera_Login_Data.sqlite	0ace9ff85bc53be1debb74c7f6a767babfef479921cbc174496e701afd2239a9
Opera_Visited_Links	8202afb985b0f213da84ea6f66f3d3351ea1372811d79802ec93d5db7322129d
Opera_Bookmarks.json	8b3fb0636a3aff6e901070e960b3f88b4e0ca2005c0be4aad16e77a8953151be
Opera_Session	c2ff247fadc6cfba55ff721d921abca5001da4e8c782dd80897635ab9c54d659

Stego carriers inside `stash.zip`. The nine members of the password-protected archive (`stash.zip`, see Section 2), extracted after the passphrase was derived. The archive itself is not re-submitted: it is already catalogued as a recovered artifact (Section 1) and its SHA-256 is unchanged.

Carrier file	SHA-256 Value
blank	23cd980099fab1c0b6993a9d27899f8679ffe42d423d8d51b29c655c0d53bbec
polish_cow.webp	857f75db04db84340d4efa53061ca26fc7c7e6f65f80899c57f8d65b4c9d5678
myzip.zip	9ddeef450f4307a974d07516519d076429b9bd1a32e192816ae0217cc8139de4
subtitles.srt	a37b26392800891fd2c29cd59f036e5a01789d6ec71fd24f22baa776e6729fe7
Grand Theft Auto VI Trailer 1.mp4	5d3b5f08f35c5d812e0f2c36d58c9c4f2a0b844e2e1f8bd5054a9bc54f8471ca
ESP32_ADXL345_3Axis_Telemetry.xlsx	b3cc6cb719af6ea2f2653b8635e5a0eabf4ae3b32b207aa01dbd5c4d176ee15f
gta_v_iceberg_unexplained.jpg	7d0f0c05c98fe80513e8e311918298f8539835d960e7270185ed25a5163d173d
dj_cara_after_hours.wav	2b61eecffc5261dcc05d833c64454b42d33f445d3df09095988d0c2dbbc8753d
[Erai-raws] Shingeki no Kyojin - 01 [720p][Dual Audio].mkv	9de6f912cfaffcb8d68f4c2db39dff1e22994c92e56e8daac70e542caf67b9b7

Supporting outputs: Command outputs produced during analysis and submitted as auxiliary items. These are re-runnable and carry a generation timestamp, so their integrity hashes live in the bundle's `MANIFEST.csv` rather than in this report.

Item	Substantiates
Windows laptop (`windows/`)	
`00_tool_versions.txt`, `00_plugin_sha256.txt`	Analysis environment; hash of the third-party BitLocker Volatility plugin
`02_pstree.txt`	Process tree at capture time; no `explorer.exe`, WinRM chain
`14_mmls_windows.txt`, `15_bootsectors.txt`, `16_key_tests.txt`	Partition layout, `-FVE-FS-` BitLocker signature, FVEK key test
`22_opera_history.txt`	Opera GX browsing timeline (phishing searches, `emkei.cz`)
`30_pypykatz_registry.txt`, `31_secretsdump.txt`	`johnny` NT hash and SYSTEM boot key
`32_software_winlogon_lsval.txt`	Autologon enabled; `DefaultPassword` = `fsociety`
`34_passphrase_derivation.txt`	SHA-256 of the five clues → archive passphrase
`35_C_hive_listing.txt`	Registry hive inodes on `C:` used with `icat`
`36_voice_ffprobe.txt`, `confession.txt`	Voice message duration (75.9 s) and transcript
Linux workstation (`linux/`)	
`40_mmls_linux.txt`	Linux partition layout (ext4 at sector 2203648)
`42_fls_deleted.txt`	`srm` secure-deletion / anti-forensic activity
`44_stash_7z_slt.txt`	`stash.zip` member listing; ZipCrypto/Deflate

Item	Substantiates
`45_ssh_key_fingerprints.txt`	SSH public-key fingerprints quoted in the report
`48_journal_sha256.txt`	System-journal integrity (per-`.journal` hashes)
`git_log.txt`	Commit provenance; preserved history after server `cleanup`
`peixinho_.git_config`	Local clone’s only remote is the live phishing site